INTEGRATION HANDOFF / PLATFORM TEAM

Brain Document Ingestion

How the platform sends an uploaded document into Brain and reads back what Brain understood from it. The ingestion architecture is real and tested. This spec is the exact contract to build against, plus the preconditions that must land before the document path works in a hosted environment.

STAGING `https://api.sandbox.brain.fi/v1`

PROD `https://api.brain.fi/v1`

- **Status: build against staging now, promote to production after it verifies**

  The storage, normalization, ledger, policy gate, and question layers are built, tested, and deployed to staging inside the `api` service. The document extraction step that turns an uploaded file into structured data lives in a separate service that is not yet built or deployed by CI. The preconditions in Section 3 must land before an upload produces a readable obligation. Integrate against staging first. Production (`api.brain.fi`) currently runs a behind revision and additionally requires a manual promote, so do not point at it until the preconditions are promoted there. The API contract itself is stable and safe to code against today.

01 THE PIPELINE, END TO END

Six steps. The platform initiates three of them over the public API. The rest run automatically inside Brain. Each step is marked with its current deployment state.

1 **Upload the document** • DEPLOYED

`platform to brain` POST `/v1/raw/ingest` as multipart form data with the file, its mime type, and a `source_type` of `pdf_upload` or `csv_upload`. Brain stores the raw bytes

immutably and returns a `raw_id`. Content addressed, so re-uploading the same file de-duplicates.

2 **Extract text and fields** • ENDPOINT TO BE BUILT

`platform to brain` POST `/v1/raw/{raw_id}/extract`. This public trigger does not exist yet. Brain reads the stored bytes, runs deterministic text extraction (CSV, XLSX, digital PDF) then a model pass to pull the counterparty, amount, due date, and direction, and writes a parsed record. Brain does this internally by calling its own extraction service. The platform never calls that service directly.

3 **Promote to an obligation** • AUTOMATIC • DEPLOYED

`inside brain` A worker polls every 15 seconds, projects the parsed record through the canonical layer, and materializes a ledger obligation. Model-read data is written low trust: provenance `agent_contributed`, confidence capped at `0.5`. No platform action required.

4 **The gate refuses to pay on it alone** • BY DESIGN • DEPLOYED

`inside brain` A document read on its own can never authorize a payment. The payment gate reads the ledger only and rejects auto-execution on low-trust document evidence until an independent source corroborates it or a human confirms. This is the safety property, not a limitation. See Section 6.

5 **Read back what was found** • DEPLOYED

`platform to brain` GET `/v1/ledger/obligations` returns the structured payables and receivables Brain extracted. This replaces the mock results screen with real data: who is owed, how much, when, and at what confidence.

6 **Ask questions about it** • DEPLOYED

`platform to brain` POST `/v1/wiki/question` answers plain-language questions grounded in those obligations: what is due, what is overdue, what is owed to a given vendor. Returns an answer plus the evidence rows it used.

02 PRODUCTION AUTH: ONE TENANT PER USER

Each end user or organization is their own Brain tenant, with data isolated by row-level security. This is the real product model, and it replaces the demo provisioning endpoint, which mints throwaway seeded tenants and is not production auth. The platform provisions and authenticates each user through the real onboarding path, then calls the document endpoints with that user's own token.

```
# once per user, at signup
POST /v1/signup            { email, password }        → { tenant_id, user_id }
POST /v1/auth/verify-email { tenant_id, token }       → { verified: true }
# each session
POST /v1/auth/login        { email, password }        → { token }  (this user's tenant
```

SCOPES THE USER TOKEN NEEDS

- `raw:write` and `raw:read` to upload and read documents
- `ledger:read` to list obligations and accounts
- `wiki:read` to ask grounded questions

THE RULES THAT KEEP THIS SAFE

- Never call the extraction service directly. It is internal, separately authenticated, and not exposed. Extraction is orchestrated by Brain behind the public `/v1/raw/{id}/extract` trigger.
- Treat extracted data as advisory. It is capped at **0.5 confidence** and cannot move money on its own.

Note: the owner token issued by `/v1/auth/login` does not carry `raw:write` today. Brain-core adds it to the owner scope set as part of the preconditions in Section 3, so a user can ingest into their own tenant.

03 PRECONDITIONS BEFORE THE DOCUMENT PATH WORKS LIVE

These are brain-core tasks, not platform tasks. Until they land, an upload on staging stores the file but does not produce an obligation. The platform can build and test its calls against the contract in parallel.

Deploy the extraction service BRAIN-CORE

The document extractor exists and its unit tests pass, but CI does not build or deploy it. Add it to the image build matrix and the staging and production deploy steps, with its model API key and the shared signing secret provisioned.

Add the public extraction trigger BRAIN-CORE

Step 2 above. A public `POST /v1/raw/{id}/extract` route on the api service that loads the stored bytes and calls the extraction service internally over a signed request. Nothing triggers extraction today.

Grant users the ingest scope BRAIN-CORE

The owner token from `/v1/auth/login` does not include `raw:write` or `raw:read` today, so a user cannot upload into their own tenant. Add both to the owner scope set. Enable self-serve signup in production (it is flag-gated) and confirm email verification is deliverable there.

Add OCR for photos and scans BRAIN-CORE

In v1 scope. Digital PDFs, CSV, and XLSX extract today. Photographed and scanned image-only documents need a vision model step in the extraction service. Until it lands, those files return a clear unsupported-type response rather than a wrong reading.

Promote to production BRAIN-CORE

Production runs a behind revision. A merge to main reaches staging automatically but production requires a manual promote. Verify the whole chain on staging first, then promote. Confirm each route returns non-404 on the target host before wiring it.

04 ENDPOINT CONTRACTS

POST `/v1/raw/ingest` — scope **raw:write** / ● DEPLOYED

BODY	multipart form data: `source_type` (`pdf_upload` or `csv_upload`), `file` , optional `mime_type` . Max 50 MB.
RETURNS	`{ raw_id, sha256, deduplicated }` . Status 201 for a new artifact, 200 if the identical file already existed.

POST `/v1/raw/{raw_id}/extract` — scope **raw:write** / ● TO BE BUILT

BODY	`{}`. The raw id in the path is sufficient. Brain resolves the mime type from the stored artifact.
RETURNS	`{ parsed_id, confidence }`. Idempotent: re-posting for the same artifact returns the existing parsed record.

GET `/v1/ledger/obligations` — scope **ledger:read** / ● DEPLOYED

QUERY	optional `status` (`upcoming`, `due`, `overdue`), `type`, `due_before`.
RETURNS	`{ obligations: [ { id, type, direction, amount_due, currency, due_date, status, counterparty_id, provenance, confidence } ] }`

POST `/v1/wiki/question` — scope **wiki:read** / ● DEPLOYED

BODY	`{ question: "what do I owe and when is it due" }`
RETURNS	`{ answer, evidence: [...] }`. The answer is grounded in the ledger obligations above, with the evidence rows it drew on.

05 WHO BUILDS WHAT

PLATFORM BUILDS

- The four public API calls above, in sequence.
- Replace the mock reading and results screens with real state: poll on `raw_id`, then render **ledger obligations**.
- Show **confidence** on extracted items and label them advisory. Never present a document read as ready to pay.
- Provision a Brain tenant per user via **signup, verify, login**, and keep uploads scoped to that user's token.

BRAIN-CORE SHIPS FIRST

- Build and deploy the **extraction service** in CI, staging and production.
- Add the public `/v1/raw/{id}/extract` trigger.
- Add `raw:write` to owner scopes and enable **self-serve signup** in prod.
- Add **OCR** for photos and scans (v1).
- **Promote to production** once verified on staging.

06 THE TRUST MODEL, SO THE PLATFORM FRAMES IT RIGHT

A document informs what Brain proposes. It never authorizes what Brain executes.

Everything a model reads out of an uploaded file is written at low trust and capped at 0.5 confidence. The payment gate reads the ledger only. It cannot read this data as authoritative, so a document read on its own can never move money. Data earns the right to drive an action through corroboration from an independent source, or through an explicit human confirm.

```
document read to proposal, corroboration or confirm to
authorization
```

For the platform this means: surface extracted obligations as insight and awareness (what is due, what is overdue, anomalies), and route any payment action through Brain's normal approval flow. Do not build a separate path from document to payment. That path would bypass the one guarantee the architecture exists to hold.

07 FIRST CALL TO MAKE

Once staging has the extraction service and the trigger, the whole loop is four requests:

```
# 1. upload
POST /v1/raw/ingest        multipart: source_type=pdf_upload, file, mime_type
  → { raw_id }

# 2. extract
POST /v1/raw/{raw_id}/extract   {}
  → { parsed_id, confidence }

# 3. read (obligation appears within ~15s)
GET  /v1/ledger/obligations?status=due
  → { obligations: [ ... ] }
```

```
# 4. ask
POST /v1/wiki/question     { "question": "what is overdue" }
  → { answer, evidence }
```

Brain Document Ingestion / Integration Spec

Verified against brain-core main. Confirm on staging before wiring production.